

Mini dispensa

Git

cos'è, la storia, i servizi, il versioning e i casi d'uso

Git è lo strumento che tiene la **storia** di un progetto: puoi tornare indietro nel tempo, sapere chi ha cambiato cosa e lavorare in gruppo senza rovinare il lavoro degli altri. Classe 3 · Informatica.

Argomenti

1. Il problema che Git risolve · cos'è
2. La storia di Git (chi l'ha inventato e perché)
3. "L'azienda Git": Git, GitHub e chi ci sta dietro
4. Quali servizi dà Git (e GitHub)
5. I concetti base e la logica (come "ragiona" Git)
6. Branch e merge
7. Git e GitHub — il flusso di lavoro (push, pull, Pull Request)
8. Il versioning: cos'è e **come si fa**, passo per passo
9. Casi d'uso con esempi di grafi
10. Glossario

1. Il problema che Git risolve

Immagina di scrivere un programma (o una tesi) e di salvare i file così:

```
progetto_finale.doc · progetto_finale_v2.doc · progetto_DEFINITIVO.doc ·  
progetto_DEFINITIVO_vero_QUESTO.doc
```

1. Non sai più **qual è l'ultima versione** né **cosa è cambiato** tra una e l'altra.
2. Se cancelli qualcosa per sbaglio, **non torni indietro**.
3. Se siete in due a modificare lo stesso file, uno **sovrascrive** il lavoro dell'altro.

Git risolve tutto questo: tiene **una sola cartella** con dentro **tutta la storia** delle modifiche. In ogni momento puoi vedere com'era il progetto ieri, la settimana scorsa, un mese fa — e tornarci.

In una frase: Git è un **sistema di controllo di versione** (*version control*): registra ogni modifica ai file e ti lascia navigare avanti e indietro nel tempo del progetto.

2. La storia di Git

Git non è nato in un'azienda: è nato per **necessità**, da un problema molto concreto.

1. **2005.** Lo crea **Linus Torvalds** — lo stesso che aveva creato **Linux** (il sistema operativo). Gli serviva uno strumento per gestire il codice del **kernel Linux**, a cui lavoravano **migliaia di programmatori** sparsi nel mondo.
2. **Perché proprio allora.** Fino a quel momento usavano un programma di un'altra azienda (BitKeeper) che a un certo punto non poterono più usare gratis. Invece di arrendersi, Torvalds **ne scrisse uno**

nuovo in pochi giorni.

3. **Gli obiettivi** erano chiari: doveva essere **veloce**, **affidabile** (non perdere mai i dati) e capace di gestire **tanti rami di lavoro in parallelo**.
4. **Oggi**. Git è diventato lo **standard mondiale**: lo usano dalle piccole scuole alle più grandi aziende di software del pianeta.

Curiosità sul nome: in inglese britannico "git" è una parola scherzosa per dire "testa dura / persona antipatica". Torvalds disse ridendo di aver chiamato così i suoi progetti (prima Linux, poi Git) "come se stesse dando un nome a sé stesso".

Da ricordare: Git è **gratuito** e **open source** — il suo codice è pubblico e chiunque può leggerlo, usarlo e migliorarlo. Nessuno "possiede" Git.

3. "L'azienda Git": Git, GitHub e chi ci sta dietro

Qui c'è un punto che confonde tutti all'inizio, quindi mettiamolo in chiaro: **Git** e **GitHub** sono **due cose diverse**.

	Git	GitHub
Cos'è	Lo strumento (il programma / il metodo) che tiene la storia.	Un sito web che ospita online i progetti gestiti con Git.
Chi c'è dietro	Nessuna azienda: è open source, nato da Linus Torvalds e portato avanti da una comunità di volontari.	È un' azienda vera e propria (GitHub, Inc.), fondata nel 2008 e dal 2018 di proprietà di Microsoft .
Dove sta	Sul tuo computer (o come concetto).	Su Internet (i tuoi progetti "nel cloud").

Analogia: Git è come il **motore** di un'automobile (la tecnologia che fa funzionare tutto); GitHub è come un **grande garage/parcheggio online** dove tieni le tue auto, le mostri agli altri e ci lavorate insieme. Puoi usare il motore anche senza quel garage — ma il garage lo rende comodo.

GitHub **non è l'unico** garage: esistono anche **GitLab** e **Bitbucket**, che fanno una cosa simile. Noi useremo **GitHub** perché è il più diffuso.

4. Quali servizi dà Git (e GitHub)

Mettendo insieme lo strumento (Git) e il sito (GitHub), ecco cosa ottieni:

Servizio	A cosa serve
Storia completa	Ogni modifica è registrata: puoi tornare a qualsiasi versione passata.
Ritorno indietro	Hai rotto qualcosa? Torni all'ultima versione che funzionava, senza panico.

Lavoro in parallelo (branch)	Provi idee nuove su un ramo separato senza toccare la versione buona.
Collaborazione	Più persone lavorano sullo stesso progetto; Git unisce i contributi in modo ordinato.
Copia di sicurezza (backup)	Il progetto è anche online: se si rompe il PC, non perdi niente.
Chi ha fatto cosa	Per ogni riga si sa chi l'ha scritta e quando (utile nei gruppi).
Versioni pubblicate (Release)	Pubblichi una versione "ufficiale" e stabile del tuo lavoro (es. <code>v1.0</code>).
Sito gratis (GitHub Pages)	GitHub può pubblicare online una pagina o un sito direttamente dal tuo progetto.

5. I concetti base e la logica

Repository (repo)

È la **cartella-progetto** gestita da Git: contiene i file **e la loro storia completa**. Creare un repository = dire a Git "da qui in poi tieni la storia di questa cartella".

Commit

Un **commit** è una **fotografia** salvata del progetto in un certo momento, con un **messaggio** che spiega cosa hai cambiato (es. *"aggiunto il menu iniziale"*).



Una catena di commit: ognuno è un salvataggio con messaggio. Puoi tornare a qualsiasi punto.

La logica: come "ragiona" Git

Capire questa parte rende tutto il resto facile:

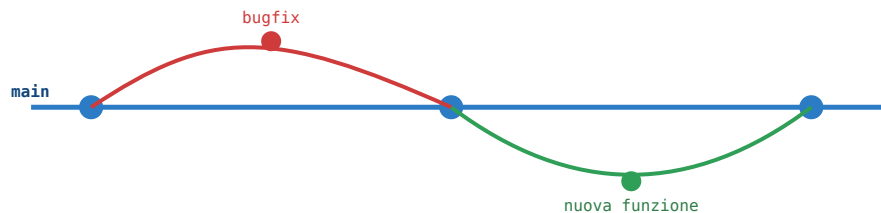
1. **Git salva fotografie, non differenze.** Ogni commit è lo stato dell'intero progetto in quel momento (come una foto). Non devi ricostruire niente: la foto c'è già.
2. **Ogni commit "punta" a quello prima di lui.** È come una **catena**: ogni anello sa qual è l'anello precedente. Seguendo la catena all'indietro, Git ricostruisce tutta la storia.
3. **Da qui nascono i rami.** Se da un anello partono **due strade**, hai due rami (branch) che vanno avanti in parallelo. Quando li riunisci, fai un **merge**.
4. **Niente si perde.** Un commit fatto resta lì: anche se cambi idea, la vecchia strada esiste ancora e ci puoi tornare.

In parole semplici: pensa alla storia del progetto come a una **metropolitana**. Ogni fermata è un commit. Di solito vai dritto, ma a volte parte una **linea nuova** (branch) che poi può **ricongiungersi** (merge). La

mappa completa non si cancella mai.

6. Branch e merge

Il **branch** (ramo) è una **linea di lavoro parallela**. Serve per provare una funzione nuova o correggere un bug **senza toccare** la versione stabile (il ramo principale, `main`). Quando la cosa funziona, si **unisce** al `main` con un **merge**.



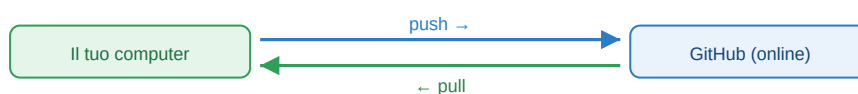
Rami che partono dal `main`, lavorano a parte e poi si fondono di nuovo nel `main`.

Perché è potente: più persone (o più idee) vanno avanti **in parallelo**; il `main` resta sempre funzionante finché non si decide di unire.

7. Git e GitHub — il flusso di lavoro

Il giro tipico

1. **clone**: scarichi il repository dal sito sul tuo computer (la prima volta).
2. **modifichi** i file e li salvi normalmente.
3. **commit**: fotografi le modifiche con un messaggio.
4. **push**: mandi i tuoi commit **online** (su GitHub).
5. **pull**: scarichi le modifiche fatte dagli altri per restare aggiornato.



Pull Request (PR)

Su GitHub, quando vuoi **proporre** di unire il tuo ramo al progetto, apri una **Pull Request**: gli altri la **guardano**, commentano, e se va bene si fa il **merge**. È il modo con cui i team lavorano insieme in modo ordinato: prima si guarda, poi si unisce.

Noi useremo tutto dal browser (dal sito GitHub, cliccando): niente comandi da digitare. I nomi `commit`, `push`, `merge` restano gli stessi, ma li fai con i bottoni.

8. Il versioning: cos'è e come si fa

Quando un progetto raggiunge un punto "buono", gli si dà un **numero di versione**. Lo standard più usato al mondo si chiama **versionamento semantico** e ha la forma **vX.Y.Z** :

Cifra	Nome	Quando aumenta
X	major	Cambiamento grosso , spesso incompatibile col passato (es. tutto rifatto).
Y	minor	Nuova funzione , ma le cose vecchie funzionano ancora.
Z	patch	Solo correzioni di bug, niente di nuovo.

Esempio (un gioco): **v1.0.0** giocabile → **v1.1.0** (aggiungi i suoni) → **v1.1.1** (correggi un bug) → **v2.0.0** (nuovo motore, tutto diverso).

Come si fa, passo per passo

1. **Lavori** normalmente e fai i tuoi **commit** (i salvataggi con messaggio).
2. Quando il progetto è **pronto e provato**, decidi il numero: parti da **v1.0.0** .
3. Metti un **tag** (un'etichetta) su quel commit con il numero di versione: è come piantare una bandierina che dice "questa è la v1.0.0".
4. Su GitHub pubblichi una **Release**: quella versione diventa **ufficiale e "congelata"** (non cambia più).
5. Continui a lavorare. Alla modifica successiva **aumenti (bumpi) il numero** secondo la regola sopra: una correzione → cambia la Z; una funzione nuova → cambia la Y; una rivoluzione → cambia la X.

Regola d'oro: un numero di versione **già pubblicato non si riusa mai**. Se cambi anche solo una cosa, prima **aumenti il numero**, poi pubblichi. Così due versioni diverse non si confondono mai.

Per i grafi completi (branch, merge, release in team) vedi la scheda "Versioning — 5 livelli".

9. Casi d'uso con esempi di grafi

Ecco alcune situazioni reali. In ogni grafo, ogni **pallino** è un commit e la **linea** è il tempo che scorre da sinistra a destra.

Caso A — Lavoro da solo, dritto per dritto

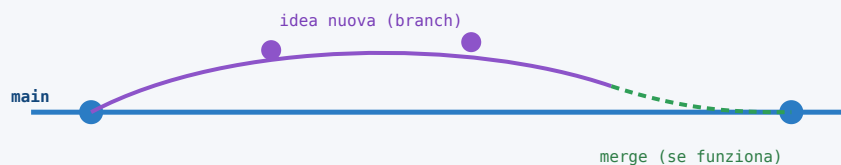
Stai scrivendo un piccolo programma da solo. Salvi passo dopo passo: la storia è una **linea dritta**.



inizio funziona migliorie finito

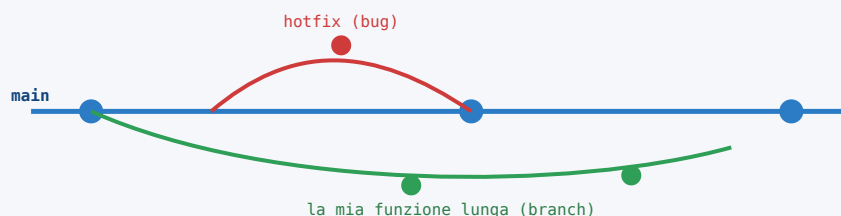
Caso B — Provo un'idea rischiosa

Vuoi provare un'idea che **potrebbe non funzionare**. Apri un **branch** a parte: se l'idea è buona la **unisci** (merge); se è un disastro, **butti il ramo** e il main resta intatto.



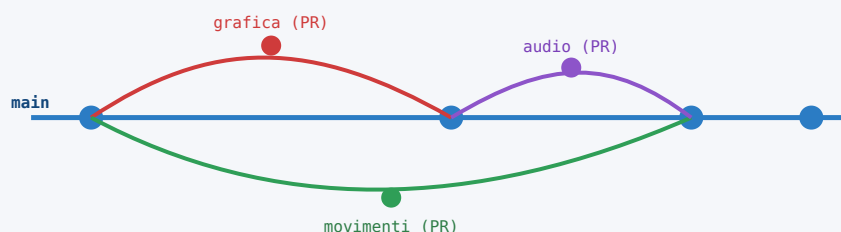
Caso C — Bug urgente mentre sto sviluppando

Stai lavorando a una funzione lunga, ma esce un **bug grave** nella versione pubblicata. Apri un ramo **hotfix** veloce, lo correggi, lo unisci subito al main — e poi torni tranquillo alla tua funzione.



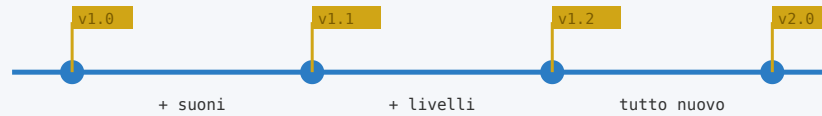
Caso D — Squadra: tanti rami e Pull Request

In un progetto di gruppo ognuno lavora sul **suo ramo** (scena, grafica, movimenti...). Ciascuno apre una **Pull Request**; il gruppo la controlla e la unisce al main. Il main cresce con i pezzi di tutti.



Caso E — Le versioni pubblicate (Release)

Man mano che il progetto avanza, si pubblicano le **Release**. Le bandierine sul main segnano le versioni ufficiali che gli utenti (o i compagni) ricevono.



10. Glossario

Termine	Significato
Git	lo strumento (open source) che tiene la storia di un progetto
GitHub	il sito (azienda di Microsoft) che ospita i progetti Git online
repository	la cartella-progetto con tutta la storia
commit	un salvataggio (fotografia) con un messaggio
branch	una linea di lavoro parallela (ramo)
merge	unire un branch a un altro
clone	scaricare un repository sul proprio computer la prima volta
push / pull	mandare / scaricare i commit da/verso GitHub
Pull Request	proposta di merge, da revisionare prima di unire
tag / Release	una versione pubblicata e "congelata"
versioning	dare un numero (vX.Y.Z) alle versioni del progetto
open source	codice pubblico, che tutti possono vedere e usare